

Part B — Production Architecture

Turning the little Amazon-checkout prototype into something a real testing platform could run on — the part of the stack a team leans on to decide what's actually safe to ship.

I wrote this the way I'd actually pitch it: opinionated, detailed enough to hand to an engineer without a follow-up meeting, and honest about the parts I wouldn't ship until I'd watched them hold up. Where a claim already has a working seed in Part A, I say so — §14 maps every production claim back to the function that already does a small version of it.

One framing up front, because it shapes the rest. I could burn this whole document picking an orchestration framework — LangGraph, a custom orchestrator, whatever — and honestly it wouldn't change much; swapping the engine that runs the agents is a contained piece of work. The two things that are genuinely painful to change once you've built on them are the **agent harness** (the runtime the agents live inside) and the **evaluation loop** (how you actually know the system is improving and not quietly rotting). So I've built everything around those two, and let agents, models, graph and guardrails hang off them.

0 · The one belief this rests on

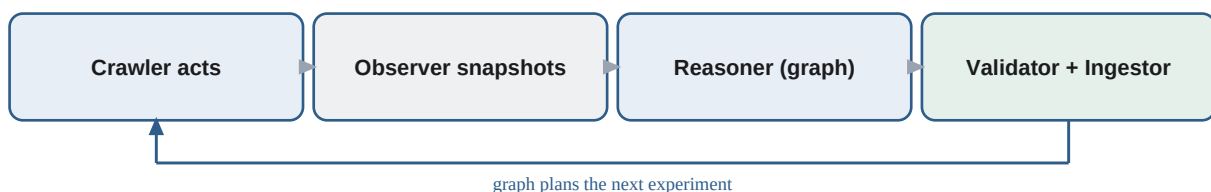
A browser agent and a knowledge graph are good at almost opposite things, and the whole design is really just about not asking either one to do the other's job.

The browser agent is opportunistic. Point it at a page and it'll work out which control adds the item to the cart — genuinely hard, fuzzy work that keeps working after Amazon reshuffles the DOM. But it only ever exercises the paths it happens to walk down, and it's shaky about whether it actually pulled something off — it'll report success on an action that quietly did nothing.

The graph is the other kind of thing entirely. It can't click anything, but it's the part that *remembers*: every state we've seen, every control, what we actually proved versus merely noticed, and — the bit that matters most — what we'd expect to be there and haven't found.

The bet this rests on: the crawler is how you discover, the graph is how you remember and cross-examine, and neither is worth much on its own. The value shows up in the loop between them — and in the harness that stops that loop from lying to itself.

The failure mode I actually lose sleep over isn't the agent missing a button. You can always crawl again. It's the system **silently believing something false** — "checkout works" when the click never landed, "the promo field is gone" when the agent just wandered. So the whole architecture is built to make confident-wrong states *expensive to reach and cheap to detect*. That single idea shows up in every section below.



The core loop: the graph plans, the browser acts, the harness validates and writes memory, and the graph reasons over what it learned — then does it again.

1 · The multi-agent network

The biggest call I made is to split the work into small, single-purpose agents with hard boundaries, instead of building one clever agent that does everything. A mega-agent that decides what to test, clicks it, judges whether it

worked, and writes the result is impossible to trust or debug — every failure is tangled up with every other one. At Testsigma's scale this becomes a network of specialised agents. Here's how I'd decompose it, with the boundary each one isn't allowed to cross:

Agent	Owns	LLM?	The boundary I enforce
Planner	The frontier — what to explore/test next, priorities, budgets	No	Decides <i>what</i> ; it never touches the browser.
Crawler	One UI action via browser-use (DOM, ARIA, screenshot)	Yes (small/fast)	Executes <i>how</i> ; it never writes graph truth or decides what "passed."
Observer	Page snapshot → typed state, controls, concepts, raw artifacts	Mostly no	Hands back the <i>raw evidence</i> (DOM, screenshot, URL), not a summary to trust.
Reasoner	Missed / absent / causal scenarios from graph structure	LLM proposes, Cypher decides	The trustworthy floor is deterministic; the LLM only <i>widens</i> the candidate set.
Validator	Deterministic post-conditions after an action	No	Nothing becomes "validated" until it re-checks from independent evidence.
Healer	Selector repair when a locator breaks	Small model + graph	Proposes a fix; the Validator still has to confirm it.
Triage	Cluster failures, decide flake vs. real regression	LLM + graph history	Escalates; it doesn't silently suppress.
Safety supervisor	The guardrails, checked before every action	No	Vetoes <i>before</i> the browser ever sees the task.
Ingestor	Idempotent graph writes with provenance + confidence	No	No model calls; every write is typed and replayable.

The boundary I care about most is between the Crawler and the Validator. **The Crawler is allowed to be wrong. The Validator's entire job is to not believe it.** When the Crawler says "I changed the quantity," the Validator re-reads the actual cart and checks the number moved — because the model is wrong about exactly that surprisingly often.

The other thing worth spelling out is what happens when an agent **fails, stalls, or returns low confidence** — because that's where demos and production part ways:

- **Fails** (throws, times out): the harness retries with backoff up to a budget, then marks the step failed and moves on. A failed step is data, not a crash.
- **Stalls** (keeps acting without progress — I actually hit this: the agent re-clicked "increase quantity" four times): a per-run action ledger + a no-repeat guard turns it into a fast veto instead of a runaway loop.
- **Low confidence**: the agent doesn't get to quietly pass. Below its threshold it either escalates to a stronger model, hands off to a human, or writes an explicit "unverified" marker into the graph. Uncertainty is a first-class output, not an inconvenience.

1.1 • The contract between agents

Agents don't share a blob of state — they pass **typed contracts** (Pydantic schemas in Python), and that's deliberately the most important interface in the system. Every hand-off carries: the intent, the expected state it applies to, a **confidence structure** (not a vibe — a number plus what it's conditioned on), success evidence to look for, a risk class, and an escalation signal. I'd freeze those contracts before a single agent is built, because they're what lets me swap the model or the framework underneath any one agent without the others noticing.

1.2 · How the Validator checks actions it has no rule for

There is a fair question hiding in all of this. The Validator can only confirm things it knows how to check. So what happens when an agent does something real and useful, but the Validator has no rule for that exact action?

The short answer is that it never pretends. Every action lands in one of three states, not two:

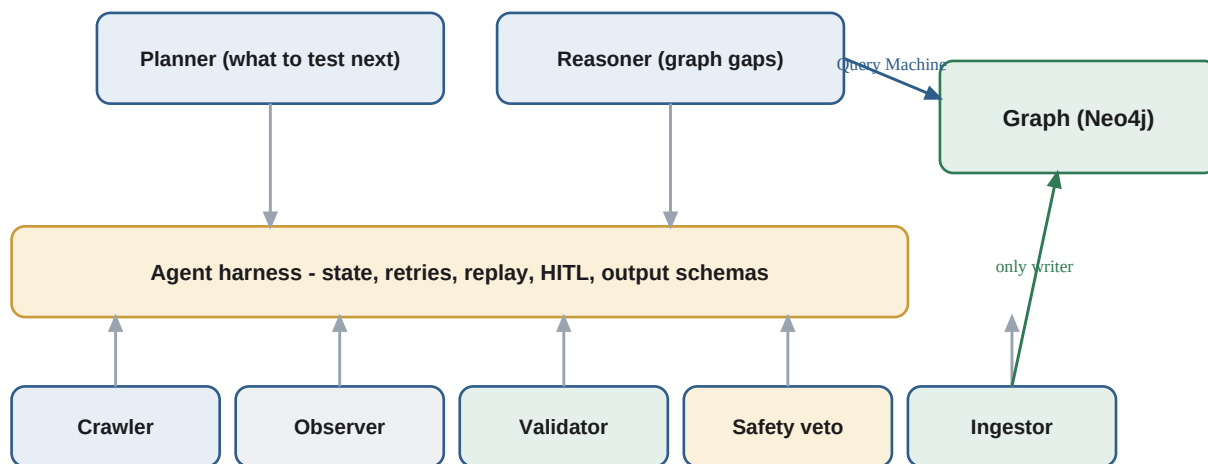
- **Done and confirmed.** The Validator checked, and the effect really happened. This is the only state we call "validated."
- **Done but not confirmed.** We know the click landed, but we have not proven what it did. We label this honestly as "unverified." We do not call it a success.
- **Blocked or failed.** The action did not run at all.

So a legitimate action the Validator cannot check is not thrown away, and it is not marked wrong. It is kept and marked "unverified." The system would rather admit it is not sure than claim a win it cannot back up. For a tool whose whole job is telling you what is safe to ship, that is the safe direction to be wrong in.

The next question is the real one: how does the Validator learn to check new things without a person hand-writing a rule for every single action? There are four ways, listed from the one I trust most to the one I trust least.

1. **The graph tells it what to expect.** The graph already stores cause-and-effect rules, like "changing the quantity should change the subtotal." The Validator just reads that rule and checks it. Adding a new check is a small data edit, not new code. 2. **It checks things that must always be true.** Some facts hold no matter what action ran. The cart total should equal the sum of the items. You should not be able to reach checkout with an empty cart. A good action keeps these true. A bad one breaks one, and we catch it. 3. **It compares the page before and after.** Take a snapshot before the action and one after. If nothing changed when something clearly should have, that is a red flag, and you do not need a special rule to notice that nothing happened. 4. **It asks a second model to be the judge — for the truly unknown.** When there is no rule, no invariant, and no clear before-and-after signal, we fall back to an LLM acting as a judge of whether the action did what it was supposed to. Two things keep this honest. First, the judge is a *different* model from the one that took the action, so a model is never grading its own work. Second, we regularly check how often the judge agrees with a human, so we know exactly how much to trust it.

The idea that ties all four together is simple: **validation is not a yes/no stamp, it is a confidence score.** A deterministic check gives high confidence. The LLM judge gives medium confidence, and we flag it. No check at all means the action stays "unverified" with low confidence and gets sent to a human or added to a saved test set. Nothing is ever quietly treated as true. The system is always allowed to say, out loud, that it is not sure.



Only the Ingestor writes graph truth; nothing is 'validated' without the Validator.

The agent network around the harness and the graph. Nothing writes graph truth except the Ingestor, and nothing becomes "validated" without the Validator.

2 · The agent harness — this is the architecture

If I could only get one thing right, it'd be this. The **harness** is the runtime every agent runs inside. It owns state, retries, timeouts, tool dispatch, output-schema enforcement, deterministic replay, and the human-in-the-loop interrupt protocol. Frameworks like LangGraph give you the state-machine primitives; the value I'm adding lives *above* them.

What the harness owns, concretely:

- **State + replay.** Every run is a durable, resumable workflow — I'd use **Temporal** here, because crawls are long-running and I want to replay a failure step-by-step six weeks later, not guess at it. Every agent step is recorded with its inputs, outputs, model + prompt version, and a trace ID.
- **Structured tool dispatch + output validation.** Agents don't return prose that something downstream parses hopefully. They return schema-validated objects, and a response that doesn't fit the schema is a failure the harness catches — not silent garbage that becomes ground truth.
- **Retries, timeouts, circuit breakers.** Per-stage latency and token budgets. A stage that blows its budget surfaces as an observable failure, not silent spend or a hang.
- **HITL interrupts.** A clean protocol for "stop and ask a human" — with the run paused, not abandoned, so a person can answer and the workflow continues.

This is the thing I'd actually stand behind in a design review: which orchestration library sits underneath is close to interchangeable. What's expensive to get wrong — and expensive to unpick later — is the set of contracts the harness enforces: what state looks like, how a run replays, how an output gets checked, when a human is pulled in. So that's what I'd design and defend before anything else.

3 · Guardrails, built into each agent — not one shared layer

Agent systems fall over in ways a demo never shows you, and bolting one shared "safety layer" across the whole thing is the wrong shape — each agent is unsure about completely different things. So I'd attach guardrails to each agent individually: every agent says what "not confident enough" means for *it*, and what to do the moment it crosses that line.

The Part-A prototype already ships the sharpest example of this — the **deny-list safety veto**. I want to defend the shape of it, because it's the opposite of the obvious choice.

The tempting move is an **allow-list**: enumerate the safe actions, permit only those. Safe, and wrong for *this* system. The whole reason to put a graph behind the crawler is to surface scenarios nobody pre-conceived — an allow-list can only permit actions someone already thought of, so it would block every newly discovered or graph-inferred intent and quietly defeat the product. A guardrail that only passes what you already enumerated can't guard a system whose job is to find what you *didn't* enumerate.

So safety is a **deny-list**: default-allow, block a short, enumerable set of known-bad actions before they execute. The trick is that the genuinely dangerous actions are exactly the ones you *can* enumerate, because they're irreversible or identity-level — place order, pay, sign out, navigate off the product under test. Everything reversible (quantity, delete, save-for-later, promo) is fair game, because a mistake there is cheap and the Validator catches it.

Around that, the production guardrail layer per agent:

Guardrail	What it does
Output validators / schema enforcement	The agent's output must fit its contract or it's a failure, full stop.
Confidence gating	Below threshold → escalate, fall back, or write "unverified." No silent low-confidence passes.
Circuit breakers	An agent that fails N times in a row on a feature gets tripped and the feature is flagged, not hammered.
Fallback chains	Frontier model → small model → deterministic rule → human, in that order, per agent.
The deny-list veto	The one hard wall: irreversible actions blocked before execution, as a <i>separate</i> process a planning bug can't bypass.

4 · Model routing and composition

I'm not training anything here — the whole job is picking the right model for each task and combining them without one model's guess turning into the next one's fact. So I'd keep a routing table, and I'd argue for it with eval numbers rather than brand preference. The principle is boring and correct: **deterministic by default, small models for high-volume fuzzy work, the frontier model only where reasoning value is high and volume is low.**

Task	What I'd route it to	Why
State resolution, validation, inference, ingestion	Deterministic code (free)	~90% of operations. Must be explainable and repeatable. You never use a model to check a model.
Which element to click (the Crawler)	Small/fast — GPT-4o-mini / Haiku / Gemini Flash	High volume, low stakes, latency-sensitive UI skill.
Locator-stability / flake classification	Fine-tuned small open model (Mistral-class)	Narrow, repetitive, cheap to fine-tune, and it beats a frontier model on cost at this volume.
Graph gap reasoning, root-cause chains, eval adjudication	Frontier — Claude-class, extended thinking	Low volume, high reasoning value; amortised across customers and releases.

A few opinions I'd stake:

- **Chain models without letting errors snowball.** The danger in a pipeline is one

model's guess becoming the next model's "fact." The defence is the same as everywhere else here: between model stages sits a deterministic check or a graph lookup, so a guess never silently hardens into truth.

- **Pin prompt + model versions.** Every run records which model and prompt version ran.

When a provider ships a "better" model, I want to know it changed *my* decision quality before a customer files a ticket — which is a canary-eval problem (§8), not a hope.

- **Context management for long sessions.** A long agentic crawl accumulates context

fast. I'd compress aggressively: the graph *is* the long-term memory, so the working context stays small — the agent pulls exactly the graph slice it needs via the Query Machine (§6) rather than dragging the whole history along.

On cost — the brief asks me to show the math, so here it is. Two model tiers, with rough late-2025 per-million-token prices: the **small model** (GPT-4o-mini / Haiku class) at about \$0.15 in / \$0.60 out, and the **frontier model** (Claude / GPT-4o class) at about \$3 in / \$15 out.

Unit	What runs	Tokens	Cost
Per feature-crawl — the crawler	~25 small-model steps @ ~3k in / 0.3k out	75k in / 7.5k out	~\$0.016
Per feature-crawl — graph reasoning	~4 calls at convergence, mostly cache hits	16k in / 2k out	~\$0.004 (→ ~\$0 cached)
One full crawl			~\$0.02
Per graph query — tier-1 template	deterministic Cypher, no model	0	~\$0
Per graph query — tier-2 NL→Cypher (<5% of queries)	one frontier call	2k in / 0.3k out	~\$0.01
Per customer / month — crawls	50 features × 30 nights = 1,500 crawls		~\$30
...with caching (~50% of pages unchanged night-to-night)	fewer live model calls		~\$18–20
Frontier, offline (rule discovery, eval, canary)	platform-wide, amortised over customers		~\$5 / customer
All-in inference, per customer / month			~\$25

The shape matters more than the exact cents: a customer crawling 50 features every night lands around **\$25 a month in model spend**, and it stays there for three reasons — the cheap model does roughly 95% of the calls, graph queries are mostly free deterministic Cypher, and the frontier model never touches the per-crawl path (it runs offline and gets amortised across everyone). The three levers that hold the line: cache model calls by graph-state signature so an unchanged page costs nothing, invalidate only the bounded subgraph a PR touched instead of re-crawling the world, and keep the frontier model off per-crawl work entirely.

5 · The production graph schema

The textbook hierarchy for UI testing is Element → Component → Flow → Feature. I wouldn't use it as the backbone, because it models the UI's *shape* but not the two things this platform actually sells: **provenance** (why do we believe this?) and **absence** (what should exist but doesn't?). So I built the graph around the questions we have to answer, not around the DOM tree.

Node types

Node	Key properties	A query it answers
------	----------------	--------------------

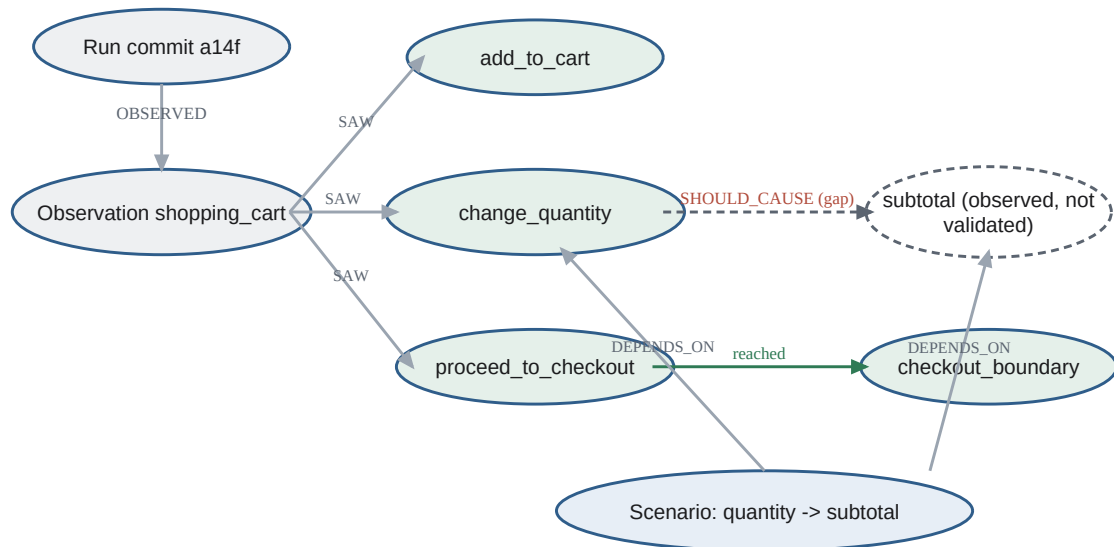
Tenant / App / Feature	ids	Everything is scoped; one feature modeled deeply.
Run	commit_sha, started_at, model_version, cost	What did we know at run N vs. N+1?
Observation	state, url, artifact_ref, screenshot_ref	Replay exactly what the browser saw.
Concept	key, kind, observed, validated, expected, confidence, first_seen, last_seen	Expected-not-observed; observed-not-validated.
Intent	source, status, risk, selector_ref, confidence	Why an action ran, passed, or was vetoed.
Scenario	key, status, confidence, last_confirmed_run	Which scenarios are trusted right now vs. decayed.
Selector	hash, css/xpath/role, stability, last_failed	Blast radius and self-healing.
CodeArtifact	path, symbol, commit_sha	Map a PR change to selectors → concepts → scenarios.

The single most important choice: **the unit of meaning is a Concept — a behaviour — not a DOM element.** Selectors are the least stable thing in the whole system; they change every redesign. Model the graph around *what a control means* ("this is the add-to-cart capability") instead of *how to find it* (#some-id), and the graph stays meaningful across redesigns while selectors churn underneath as replaceable properties.

The edges are where most of the value lives — and I'd hang two things off them that usually only get put on nodes: where the edge came from, and how sure we are of it.

Edge	Between	What it lets me ask
OBSERVED / SAW_CONCEPT	Run → Observation → Concept	"What did we see, and where?"
TARGETS	Intent → Concept	"Which attempt exercised which behaviour, and did it pass?"
DEPENDS_ON	Scenario → Concept	"Which concepts does this scenario rely on?" (blast radius)
SHOULD_CAUSE	Concept → Concept	"This cause should produce this effect" — the causal expectation the reasoner checks
BACKS	CodeArtifact → Selector	"Which code backs this locator?" (PR risk)

Here's a concrete instance of the graph after one checkout crawl — small enough to read, real enough to reason over:



A sample knowledge graph after one Amazon-checkout crawl. Solid nodes were validated; the dashed subtotal node was observed but never proven; the dashed SHOULD_CAUSE edge is the causal gap the reasoner surfaces.

That dashed SHOULD_CAUSE edge from *change-quantity* to *subtotal* — observed cause, expected effect, never proven — is exactly the kind of missed test the graph surfaces and the crawler would never have written for itself.

5.1 · Why a graph and not just vectors

I'd push back hard on "just use RAG for this." Vector similarity is great at "find me things like this," and useless at the three things that actually matter here:

- **Absence.** "What test *should* exist and doesn't" is not a similarity query — there's nothing to be similar to. It's a structural query over expected-but-not-observed, which the graph answers in one hop and a vector store can't express at all.
- **Temporal reasoning.** "What did the system know at commit X?" needs edges stamped with time and commit_sha, not nearest neighbours.
- **Multi-hop traversal.** CodeArtifact → Selector → Concept → Scenario is a four-hop walk. Vectors give you one hop of fuzzy similarity; the walk is the whole point.

Vectors still earn a place — selector repair and semantic matching — so I'd run **pgvector** (or Weaviate) alongside Neo4j as a **hybrid**: the graph is the source of truth, the vector index is an auxiliary lookup. It never replaces graph truth; it helps you *find* a starting node for the traversal.

5.2 · Indexes and constraints I'd actually declare

A schema without indexes is a wish. Here's what I'd declare on day one, and the query each one exists for:

```

-- identity: makes MERGE correct under concurrency, and creates the index for free
CREATE CONSTRAINT concept_id IF NOT EXISTS
  FOR (c:Concept) REQUIRE (c.tenant_id, c.feature_key, c.key) IS NODE KEY;
CREATE CONSTRAINT run_id IF NOT EXISTS
  FOR (r:Run) REQUIRE (r.tenant_id, r.app_id, r.run_id) IS NODE KEY;
CREATE CONSTRAINT selector_id IF NOT EXISTS
  FOR (s:Selector) REQUIRE (s.hash) IS UNIQUE;

-- hot-path lookups and temporal queries
CREATE INDEX concept_scope IF NOT EXISTS FOR (c:Concept) ON (c.tenant_id, c.feature_key);
CREATE INDEX run_time IF NOT EXISTS FOR (r:Run) ON (r.commit_sha, r.started_at);
CREATE INDEX code_artifact IF NOT EXISTS FOR (a:CodeArtifact) ON (a.commit_sha, a.path, a.symbol);
CREATE INDEX concept_lastseen IF NOT EXISTS FOR (c:Concept) ON (c.last_seen);

```



```
-- relationship (edge) properties for "what did we know at time T"  
CREATE INDEX edge_validity IF NOT EXISTS FOR ()-[e:SHOULD_CAUSE]-() ON (e.valid_from, e.confidence);
```

A few of these earn their keep specifically. The unique index on `Selector(hash)` is what makes blast radius and self-healing fast — you find a locator by its content, not by where it sits on the page. The `CodeArtifact(commit_sha, path, symbol)` index is the entry point for the PR hook. And the relationship index on `valid_from / confidence` is what turns the temporal question — "what did the graph believe at commit X, and how sure was it?" — into a cheap lookup instead of a full scan. The heavy blobs (DOM, screenshots) are never indexed, because they never live in Neo4j; `observation.artifact_ref` just points at object storage.

6 · The Query Machine — how the LLM actually uses the graph

A graph is only as useful as the questions the agents can ask it. Dumping the whole graph into a prompt doesn't work (context blows up, and the model drowns), and hand-writing a Cypher query for every agent need doesn't scale. So I'd build a **Query Machine**: the layer that lives between the agents and Neo4j and turns "what does the graph know?" into structured, bounded, typed answers the LLM can actually reason over.

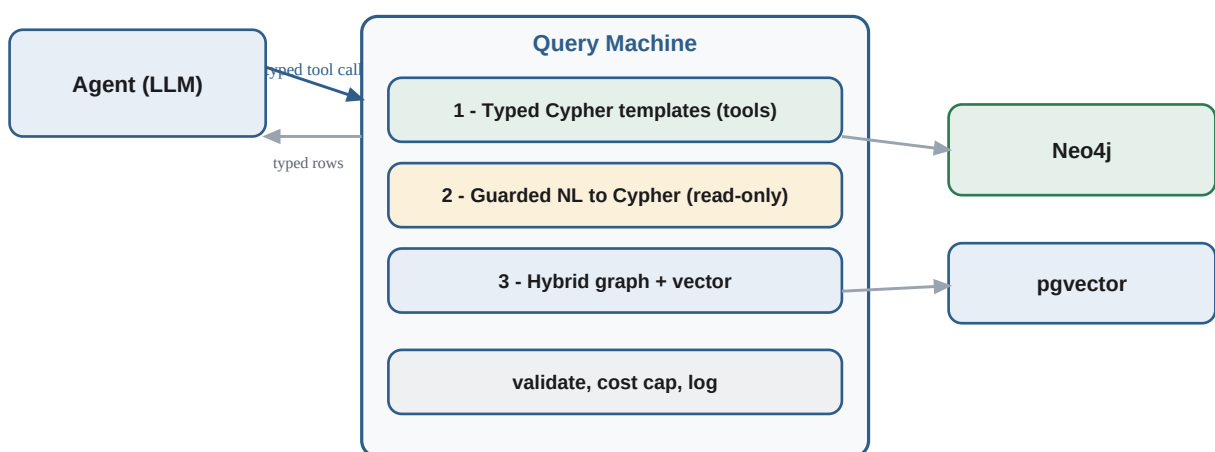
It has three tiers, and the ordering is the whole point:

1. A library of parameterised, typed graph queries — exposed as tools. These are the 90% case.

`missed_scenarios(feature)`, `absence(feature)`, `blast_radius(concept)`, `neighbours(concept, hops)`, `confidence_of(scenario)`, `what_did_we_know_at(commit)`. Each is hand-written, reviewed Cypher with a typed result schema. The agent LLM picks a query and fills the parameters as a structured tool call — it never writes Cypher. Deterministic, cacheable, can't hallucinate a traversal. **2. Guarded natural-language → Cypher, for the long tail.** When an agent has a genuinely novel question, a frontier model translates it to Cypher — but the query runs **read-only**, against a schema allow-list, with a cost/row cap, and the generated Cypher is logged for review. If it fails validation, it falls back to tier 1. The LLM gets flexibility without getting the keys to the database. **3.**

Hybrid graph + vector. For "find the right starting node" the Query Machine can hit pgvector first (semantic match on a concept/selector), then hand the node id to a graph traversal. Similarity to *locate*, structure to *reason*.

Every answer comes back the same way: typed rows, each carrying provenance and confidence, small enough to drop into the model's context. That's what "the graph information can be fully utilised by the LLM" actually means in practice — not a bigger prompt, but a **bounded, trustworthy query interface** the agent calls like any other tool.



The Query Machine: agents ask questions as typed tool calls; deterministic Cypher templates answer the common ones; a guarded NL→Cypher path handles the long tail; every answer is typed, provenance-tagged, and small enough for the model's context.

In Part A this exists in embryo: the deterministic inference (`infer_missed_scenarios`) and the blast-radius traversal are the first two "tools," and the graph-expansion LLM is the first taste of the NL path. Production is mostly turning that into a real, versioned, guard-railed query library.

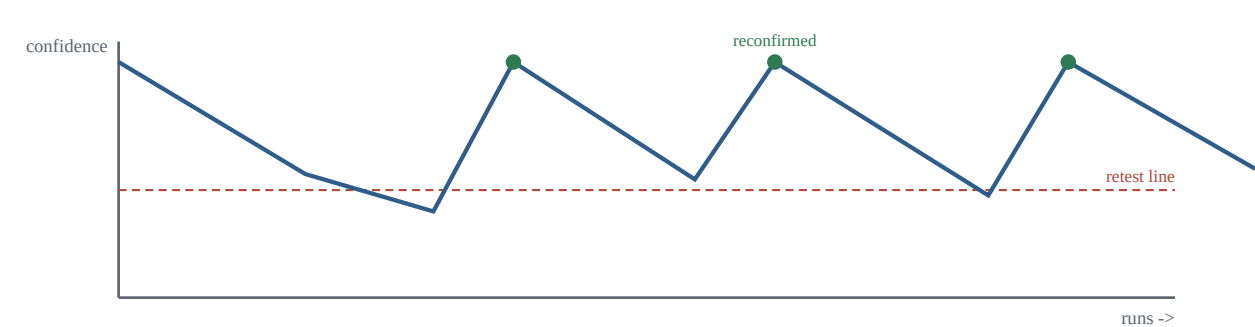
7 · Keeping the graph correct over time (the living property)

The brief pushes hardest here, rightly: how is the graph still correct on the 50th run, six months in, after 30 PRs? The answer is a clean split between what's incremental, recomputed, invalidated, and what compounds.

Operation	Trigger	How
Incremental	every crawl	Upsert observations, bump <code>last_seen</code> , append evidence, add <code>Scenario-[:CONFIRMED_BY]->Run</code> . <code>O(seen this run)</code> .
Recomputed	a run touching a feature	Re-run inference + confidence for that feature only . Never global.
Invalidated	a PR touches mapped code; a selector fails N times; repeated validation failure	Mark a <i>bounded subgraph</i> stale; push affected scenarios into the retest frontier.
Compounds	many runs over time	Selector stability, flake rate, transition probabilities, confidence-decay curves — the six-month moat a fresh competitor can't clone.

Two things I'd design carefully because they're where "living" gets hard:

- **Conflict resolution.** When this run disagrees with last week (the promo field was here, now it's gone), I don't overwrite — I keep both, stamped with `commit_sha` and time, and let confidence decay carry the old signal down. If someone asks what the system believed back at commit X, that's still answerable, because I never delete history — I supersede it.
- **Decisions become signal.** Every validated agent decision feeds the next run — a confirmed scenario raises the prior, a repeatedly-flaky one lowers it. That's the part that gets more valuable the longer the platform runs, and it's the reason the confidence-decay mechanics below aren't a nice-to-have.
Confidence decays unless something renews it — roughly, halve the distance to a floor each time a run *could* have confirmed a scenario but didn't; reset on a fresh confirmation. A scenario that drops below threshold re-enters the retest frontier on its own.



Confidence over successive runs: a scenario decays when runs don't reconfirm it, and snaps back to full confidence the moment a run proves it again. Below the retest line it re-enters the frontier automatically.

The refusal: I will not ship "recompute the whole graph nightly" as the freshness story. It's expensive and it hides drift. Bounded invalidation, proven on a golden tenant to catch a real regression, is the version I'd defend.

8 · Eval and confidence — the other spine

Honestly, this is the layer that decides whether you've built a product or a science project, and I'd put more design effort here than into any single agent. There are two places this system can be confidently wrong, and they need different checks: the agent clicked the wrong thing (perception), and the graph inferred a scenario that isn't real (reasoning).

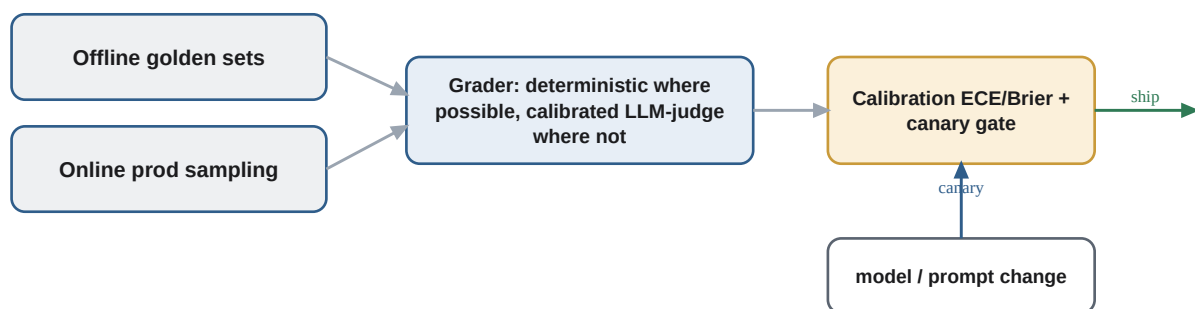
Three eval layers I'd build:

1. **Offline golden apps + regression suites.** Versioned, human-labelled trajectories. Every prompt or model change runs against them before it ships. Deterministic pass/fail plus trajectory- and step-level grading. 2. **Online production sampling.** A sampled slice of real runs graded continuously, so you catch drift the golden set didn't anticipate. 3. **Canary evals.** Every model/prompt bump runs against the golden trajectories *before* customer rollout. This is what catches a provider shipping a "better" model that quietly degrades *your* decision quality.

Question	Signal	Metric
Did it click the right target?	trace + positive/negative-target check	wrong-target rate, wander rate
Did the action succeed?	deterministic post-condition after re-observe	validation pass rate
Is the scenario reproducible?	replay across clean sessions	flake rate, pass@k
Are confidence scores honest?	predicted confidence vs. observed reproduction	ECE, Brier score
Did a model change regress us?	canary on versioned golden trajectories	delta vs. baseline

On tooling: I'd start on a managed eval store (Braintrust or LangSmith-style) to move fast, but I'd own the golden sets and the grading logic in-house, because that's the moat. **LLM-as-judge, calibrated against human spot-checks** — never an uncalibrated judge, and never a model grading something that has a deterministic check available.

The confidence point deserves its own emphasis, because I treat it as a structural requirement rather than a feature: **the system has to be honest about what it isn't sure of**. Every agent output carries a calibrated confidence, the harness gates on it, and I *measure* the calibration (ECE / Brier) rather than trusting it. Confidence without calibration is just false authority, and it doesn't go anywhere near a customer UI until the calibration check passes.



The eval spine: offline golden sets and online sampling feed a grader (deterministic where possible, calibrated LLM-judge where not); calibration and canary gates stand between a model change and a customer.

9 · Observability and operations

Agentic systems fail in ways ordinary services don't — same prompt, different outcome; a correct action the model *reports* as a failure; slow drift after a model upgrade. You can't operate that on plain logs.

- **One trace id threaded through the whole chain of agent calls**, with the model and

prompt version stamped at each hop — **OpenTelemetry** underneath.

- **Decision audit trails** — the frontier score, the guardrail decision, the confidence,

for every action that ran or was vetoed. Every decision is replayable.

- **Latency + cost budgets per agent stage**, so a stall or runaway is an observable failure, not silent spend.

- **Anomaly detection tuned for non-determinism** — the alerts I'd actually wire:

executor wander-rate climbing after a model upgrade (a canary that shipped anyway), validation-failure rate spiking on a feature (a real regression or a broken selector), stale-concept count growing without new runs (the freshness loop stalled).

And because autonomy should be *earned*, the platform moves through rollout modes rather than flipping a switch: observe-only → recommend → gated execution (safe actions auto-run, mutating ones need approval) → autonomous safe execution, only after confidence, flake rate, and safety thresholds clear.

10 · Multi-tenancy and scale

The platform has to work for a team of five and a team of five hundred, and one customer's knowledge graph must never bleed into another's. My default here is to **over-isolate**, and I'm comfortable saying so — in a tool that crawls people's apps and stores their DOM and screenshots, a cross-tenant leak is existential.

Layer	Isolated per tenant	Shared
Graph	Tenant/app/feature partition; database-per-tenant for big accounts	Schema + inference engine (code, not data)
Artifacts	Per-tenant object-store namespace, retention, redaction	Storage implementation
Models/prompts	Customer vocabulary stays isolated; versions in traces	Generic prompt templates
Learning	No selectors, DOM, screenshots, or scenarios cross tenants	Aggregated, reviewed <i>structural priors</i> only

For scale: **async agent execution** on a queue (**Celery + Redis**), long-running crawls as durable **Temporal** workflows, LLM calls cached by graph-state signature, and graph-query optimisation via the parameterised Query Machine (bounded, indexed queries instead of ad-hoc traversals). The heavy artifacts (DOM, screenshots) live in **S3/GCS**; Neo4j stores references only. Backend is **Python (async, typed) + FastAPI**, packaged in **Docker/K8s** on AWS or GCP.

Cross-customer learning is deliberately fenced. We can learn *structural priors* — "checkout flows usually have a quantity→subtotal relationship" — as reviewed, aggregated shape. We never move a selector, a DOM snapshot, or a scenario across a tenant boundary.

11 · PR blast radius

The optional PR hook isn't a separate feature — it falls straight out of the graph. A PR is a set of changed CodeArtifacts, and the answer is a downward traversal: CodeArtifact → Selector → Concept → Scenario → confidence. The discipline is in the output: it's never "rerun all the checkout tests," it's "*these* two selectors, *these* validated scenarios, and *this* one inferred scenario are at risk — retest exactly those." Part A already does the traversal (pr_blast_radius.py); production adds confidence and freshness to the ranking.

12 · The concerns the brief didn't list — but I'd raise anyway

A few things aren't on the checklist and would come up in a real design review anyway. I'd rather put them on the table myself.

Security and privacy of what we capture. We store DOM snapshots, screenshots, and session cookies — some of the most sensitive data a customer has. DOM and screenshots routinely contain personal data, and cookies *are* credentials. So: everything encrypted at rest with per-tenant keys; form values, tokens, and anything that looks like PII redacted out of the DOM *before* it's stored; screenshots retention-capped and access-controlled; cookies and auth tokens never written to logs or traces. Short retention by default, and a delete path a customer can actually trigger.

Test data and safe environments. The platform must never crawl a real production account with real user data and real money. Each tenant gets dedicated test accounts and a sandboxed or staging target, seeded with known test data. The deny-list veto is the last line of defence even there — reaching the checkout *boundary* is the assertion; crossing it is the thing we structurally forbid.

Auth and session management. The crawler has to be logged in to test anything real. Sessions are captured once — a human handles the OTP / CAPTCHA, exactly like the prototype's saved Amazon login — then stored encrypted, per tenant, and refreshed when they expire. The agent is never allowed near account-level controls (change password, sign out, switch account); that's enforced by the same deny-list that blocks payment, not by hoping the model behaves.

How it plugs into CI and the release gate. The end goal is a PR hook: a pull request comes in, the blast-radius traversal names the handful of scenarios at risk, the platform re-tests exactly those, and the result gates the merge. But I'd *earn* that gate rather than assume it — start in recommend mode (post the risk report, block nothing), move to a gating mode only once the confidence numbers hold up, and make the gate flake-aware so a known-flaky test can't block a good release. A testing platform that cries wolf gets switched off.

13 · What I'd refuse to ship — and the hardest problem

This is the section I'd most expect to get grilled on, which is exactly why it's here.

Refusal	Why, and what I'd need first
A path that can click final purchase — even behind a flag	Flags get flipped. Money pages must be <i>structurally</i> incapable of a final submit. First I'd want a red-team that tries to make it buy something and fails every time.
LLM self-report as validation	Every mutating action needs a deterministic post-condition. The prototype already does this for cart and checkout.
An uncalibrated confidence number in the UI	Confidence without calibration is false authority. Not until \$8's calibration check passes.
Nightly full-graph recompute as the freshness story	Expensive, and it hides drift. Bounded invalidation, proven on a golden tenant.
Unreviewed cross-tenant learning	Priors may cross tenants; concepts/selectors/DOM/scenarios may not. First I'd want an audit log and a human promotion step.
The prototype's hard-coded concept vocabulary, as-is	Fine for one feature, wrong for a hundred apps. Production replaces the keyword map with a learned, per-tenant concept model behind an adapter.

If you asked me for the single hardest problem in this space, I'd give you this one: **knowing the system is wrong before a human notices.** Everything downstream leans on confidence you can actually trust, and calibrating that confidence is genuinely hard when the thing grading you is itself a model and the ground keeps moving under you — a new app, a new model version, a new sprint. I don't think there's a clean solved answer yet. The best I've got is the shape in this document: deterministic floors under the fuzzy parts, confidence that's measured rather than assumed, a canary between any model change and a customer, and a graph that remembers what used to be true so you can notice the moment it stops being true. That's the problem I'd happily spend the next

few years on.

14 · How Part A grounds every claim here

None of this is hand-waving — each production claim already has a working seed in the prototype:

Production claim	Prototype seed
Planner / Crawler split; graph plans, browser acts	GraphGuidedExplorer._autonomous_loop, BrowserUseIntentExecutor
Guardrails an LLM can't bypass (deny-list, pre-execution)	safety_guard.veto_reason, in the browser-use step callback
Executor output is evidence, not truth	re-observe + _run_assertions, cart-delta / checkout verification
Graph reasons about missed + causal gaps	infer_missed_scenarios, CAUSAL_EXPECTATIONS / SHOULD_CAUSE
The Query Machine (tier 1 + NL taste)	infer_missed_scenarios, blast_radius, graph_expansion.expand_from_graph
Absence as a graph query	seed_expected_concepts, missing_expected_concepts
Everything after the graph takes over is the graph's	graph_phase_started attribution
PR blast radius as a traversal	pr_blast_radius.py, GraphStore.blast_radius
Provenance + confidence on writes	write_intent, write_observation (source, confidence, last_seen)

And to make the "not just a crawler with a database" point concrete, here's one real run, three levels deep:

- **Discovered by the crawler (it physically clicked it):** *Add to Cart works, and the quantity stepper moves.* The crawler did these, and the Validator confirmed them by re-reading the actual cart. Status: `crawl_validated`.
- **Inferred by the graph (the crawler never did it):** *Proceeding to checkout should reach the checkout boundary.* The free crawl poked around the cart and stopped — it never pushed through to checkout on its own.
- **Why the graph inferred it:** the graph holds the causal rule ``action.proceed_to_checkout SHOULD_CAUSE domain.checkout_boundary``. It had *observed* the Proceed-to-Checkout control, but no attempt had ever validated the checkout boundary. Observed cause, expected effect, no proof — that's a gap.
- **The evidence behind it:** three queryable facts in the graph — `action.proceed_to_checkout` marked observed, the `SHOULD_CAUSE` edge present, and `domain.checkout_boundary` expected but not validated. Not a hunch; a structural query.
- **What happened next:** the graph turned that gap into a directed probe, the browser clicked Proceed, and it landed on the real checkout page — which is exactly what flipped the run's end-to-end result to true. A crawler with a database would have stopped at "the cart looks fine."

15 · The stack, and why

Tools should fall out of the boundaries, not the other way round. Here's what I'd reach for and, just as important, what I'd avoid.

Layer	I'd start with	I'd avoid
-------	----------------	-----------

Agent harness	Custom harness over LangGraph primitives; Temporal for durable runs	A fully autonomous agent with no state/replay contract
Browser execution	Playwright + browser-use , sandboxed	An agent with no action contract or veto
Graph store	Neo4j (temporal patterns, provenance/confidence on edges)	Graph-shaped deps in relational tables
Retrieval	pgvector / Weaviate , auxiliary — hybrid graph+vector	Letting vector similarity replace graph truth
Query Machine	Parameterised Cypher tool library + guarded NL→Cypher	Free-form model-written Cypher on the hot path
Models	Claude (frontier reasoning/judge), GPT-4o-mini / Haiku / Gemini Flash (executor), fine-tuned Mistral-class (narrow classifiers), deterministic for validation	"We use OpenAI" as the whole model strategy
Eval	Versioned golden apps; Braintrust/LangSmith-style store; calibrated LLM-judge	Shipping prompt changes without canaries
Observability	OpenTelemetry + trace IDs + decision audit	Plain logs with no trace IDs
Async / queue	Celery + Redis; FastAPI; Docker/K8s on AWS/GCP	Fire-and-forget cron for long, retryable crawls
Artifacts	S3 / GCS ; Neo4j holds refs only	Large DOM/screenshot blobs inside Neo4j

If I had to put the whole thing in one line: the valuable part was never the clever crawler — it's the loop between something that explores and something that remembers, with a hard wall between what the system *did* and what it's allowed to *believe*. Get the harness and the eval layer right, and everything flashy underneath becomes a swappable detail.